

Laboratorio I

a.a. 2025/2026

Classi di sistema & moduli utili

Contenuti

- Oggetti predefiniti (built-in)
 - Wrapper di tipi base
 - Date
 - RegExp
 - Array con tipi di macchina
 - Collections
 - JSON
- Oggetti dell'ambiente (host-defined)
- Alcuni moduli utili (di node.js)
 - File system
 - Web server
 - Readline

Oggetti predefiniti (built-in)

Abbiamo usato sin dall'inizio del corso un certo numero di oggetti predefiniti

- nota: le classi sono funzioni che sono oggetti, quindi con “oggetti predefiniti” intendiamo anche alcune classi

Esempi:

- **Number, Math, console, Error, ...**

Lo standard JavaScript include un certo numero di altri oggetti, che sono documentati alla pagina:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Oggi faremo una *rapida carrellata* sulle *caratteristiche principali* di *alcuni* di essi!

Wrapper di tipi base

Per ogni tipo base di JavaScript, esiste una **classe wrapper** con lo stesso nome (ma iniziale maiuscola).

- Number, Boolean, String, BigInt, ...

I costruttori prendono un argomento di tipo qualunque, e provano a convertirlo nel tipo corrispondente, se possibile

- `Number("1.33") === 1.33 → true`
- `String(14) === 14 → false`
- `String(14) == 14 → true`
- `String(14) === "14" → true`
- `Boolean("hello!") → true`

La differenza fra 1 e il più piccolo numero >1

Le classi offrono costanti e metodi statici

- `Number.EPSILON → 2.220446049250313e-16`
- `Number.isFinite(3.14) → true`

Gli oggetti offrono metodi di istanza

- `Number(3.14).toFixed(4) → '3.1400'`

Mentre le costanti letterali numeriche **non** vengono automaticamente promosse ad oggetti, quelle stringa e boolean **si**

- `String(true).charAt(2) → 'u'`
- `"pippo".indexOf("i") → 1`

Date

La classe **Date** fornisce oggetti che rappresentano date (di calendario), nel range 20 Aprile 271.821 a.C. - 13 Settembre 275.760 d.C.

Il costruttore *cerca* di interpretare date espresse come stringhe, in una varietà di formati. Senza argomenti, restituisce data e ora correnti. Con numeri, li interpreta come anno, mese, giorno, ora, minuti, secondi, millisecondi.

- `Date()` → `'Thu Mar 18 2021 21:25:34 GMT+0000 (Coordinated Universal Time)'`
- `new Date()` → `2021-03-18T21:39:40.725Z`
- `new Date("19 marzo")` → `2020-03-19T00:00:00.000Z`
- `new Date(2021,3,19,14,15)` → `2021-04-19T14:15:00.000Z`

Attenzione: il mese parte da 0=Gennaio, quindi 3 è Aprile, non Marzo (sigh!)

Gli oggetti Date sono sostanzialmente numeri, espressi in millisecondi dal 1 Gennaio 1970. Quindi possiamo usare l'aritmetica.

Sia `a=new Date("19 marzo 2021")`,
`b=new Date(2021,3,19,14,15)`

- `a>b` → `false`
- `b-a` → `2729700000`
- `(b-a)/(24*60*60*1000)` → `31.59375`

Gli oggetti Date hanno numerosi metodi d'istanza: getter e setter per i vari componenti, e conversioni da/verso stringhe e numeri

- `a.toISOString()` → `'2021-03-19T00:00:00.000Z'`
- `a.toUTCString()` → `'Fri, 19 Mar 2021 00:00:00 GMT'`
- `a.toLocaleString()` → `'3/19/2021, 12:00:00 AM'`
- `a.valueOf()` → `1616112000000`

RegExp (espressioni regolari)

Le **espressioni regolari** sono un modo per esprimere *schemi di corrispondenza* su stringhe. In JavaScript, godono anche di una denotazione letterale, come quella delle stringhe, usando però // anziché "", ' ', o ` `.

- `e = new RegExp("[a-z]+")` oppure `e = /[a-z]+/`

Il metodo `e.test(s)` verifica se la stringa `s` corrisponde alla RegExp `e`.

- `e.test("pippo")` → `true`
- `e.test("4ndr34")` → `true`
- `e.test("1337")` → `false`

Il metodo `e.exec(s)` fornisce informazioni più complete; esiste l'analogo `s.match(e)` che è usato più comunemente, e `s.matchAll(e)` che restituisce un iteratore su tutti i match (è un generatore)

- `e.exec("4ndr34")` → `['ndr', index: 1, input: '4ndr34', groups: undefined]`
- `"4ndr34".match(e)` → `['ndr', index: 1, input: '4ndr34', groups: undefined]`
- `for (var m of "4ndr34".matchAll(/\d+/g)) console.log(m)`
 `['4', index: 0, input: '4ndr34', groups: undefined]`
 `['34', index: 4, input: '4ndr34', groups: undefined]`

Altri metodi su String: `search()`, `replace()`, `replaceAll()`, `split()` accettano **RegExp**

<code>/abc/</code>	A sequence of characters
<code>/[abc]/</code>	Any character from a set of characters
<code>/[^abc]/</code>	Any character <i>not</i> in a set of characters
<code>/[0-9]/</code>	Any character in a range of characters
<code>/x+/</code>	One or more occurrences of pattern <code>x</code>
<code>/x?/</code>	One or more occurrences, nongreedy
<code>/x*/</code>	Zero or more occurrences
<code>/x?/</code>	Zero or one occurrence
<code>/x{2,4}/</code>	Two to four occurrences
<code>/(abc)/</code>	A group
<code>/a b c/</code>	Any one of several patterns
<code>/\d/</code>	Any digit character
<code>/\w/</code>	An alphanumeric character
<code>/\s/</code>	Any whitespace character
<code>/./</code>	Any character except newlines
<code>/\b/</code>	A word boundary
<code>/^/</code>	Start of input
<code>/\$/</code>	End of input

Le RegExp possono avere degli ulteriori flag, per esempio:
`/.../g` = tutte le occorrenze
`/.../i` = ignora maiuscole/minuscole
`/.../m` = funziona su più linee ecc.

Array con tipi di macchina

Per implementare certi algoritmi, specialmente se è necessario scambiare dati in formati pre-determinati con altri programmi o macchine (via file o rete), è necessario poter manipolare blocchi di memoria di dimensione esattamente specificata.

- `Int8Array`, `Int16Array`, `Int32Array`, `Uint8Array`, `Uint16Array`, `Uint32Array`
- `BigInt64Array`, `BigUint64Array`
- `Float32Array`, `Float64Array`

Gli oggetti di questi tipi sono detti “Typed Array”, e offrono metodi per convertire da normali array JS a Typed Array e viceversa. Inoltre, si può accedere ai typed array con `[]` sapendo esattamente quali byte di memoria saranno letti o scritti

Collections: Set

La classe Set implementa **insiemi** di valori qualunque.

Noi abbiamo finora usato gli oggetti per implementare gli insiemi, ma... ricordate? Le chiavi devono sempre essere stringhe!

Con i Set, le chiavi possono essere qualunque valore valido in JavaScript: posso per esempio creare insiemi di funzioni, o insiemi di oggetti.

Inoltre, la classe offre alcuni metodi utili (costruttore con copia, dimensione dell'insieme, enumerazione dei contenuti, ecc.)

```
let S = new Set() o let S = new Set(a)
```

Operazioni che modificano S

- `S.add(e)` - aggiunge e a S
- `S.delete(e)` - rimuove e da S
- `S.clear()` - svuota S

Operazioni che esaminano S

- `S.has(e)` - controlla se S contiene e
- `S.size` - dimensione di S
- `S.values()` - restituisce gli e in S (iteratore)
- `S.forEach(f)` - invoca `f(e,e,S)` per ogni elemento e di S

Operazioni tra due insiemi A e B

- `A.difference(B)`, `A.intersection(B)`, e `A.union(B)` - restituiscono un nuovo insieme
- `A.isDisjointFrom(B)`, `A.isSubsetOf(B)`, e `A.isSupersetOf(B)` - restituiscono un bool

Collections: Map

Analogamente, la classe Map implementa una **mappa**, che associa chiavi (di qualunque tipo, non solo stringhe) a valori (di qualunque tipo).

Come al solito, sono implementati anche un po' di metodi molto utili per lavorare con le mappe.

```
let M = new Map() o let M = new Map(a)
```

Operazioni che modificano M

- `M.set(k,v)` - aggiunge $k \rightarrow v$ a M
- `M.delete(k)` - rimuove k da M
- `M.clear()` - svuota M

Operazioni che esaminano M

- `M.has(k)` - controlla se M contiene $k \rightarrow v$
- `M.get(k)` - restituisce v, se $k \rightarrow v$ in M
- `M.size` - dimensione di S
- `M.keys()` - restituisce le k in M (iteratore)
- `M.values()` - restituisce i v in M (iteratore)
- `M.entries()` - restituisce i $k \rightarrow v$ in M (iteratore)
- `M.forEach(f)` - invoca $f(v,k,M)$ per ogni $k \rightarrow v$ in M

JSON

JSON è un **formato di serializzazione** per oggetti JavaScript. In pratica, consente di trasformare un oggetto (qualunque) in una stringa, e viceversa.

Ha avuto grande fortuna, perché è facilmente leggibile sia dai programmi che dagli umani.

La classe JSON contiene due metodi statici:

- `s=JSON.stringify(o)` - da oggetto a stringa
- `o=JSON.parse(s)` - da stringa a oggetto

Non sempre è possibile effettuare una conversione (per esempio: riferimenti circolari, valori di tipo funzione).

Gli oggetti JavaScript "semplici" vengono detti POJO: Plain Old JavaScript Object

```
let o = {}
o.nome = "Pippo"
o["età"] = 32
for (var i=0;i<4;i++) o[i] = i*i
o.quando = new Date()
o.arr = [4,21,true]
o.obj = {    syntax: true,
           semantics: true,
           logic: "alot" }

console.log(JSON.stringify(o))
```



```
'{"0":0,"1":1,"2":4,"3":9,"nome":"Pippo","età":32,"quando":"2021-03-18T23:33:17.989Z","arr":[4,21,true],"obj":{"syntax":true,"semantics":true,"logic":"alot"}}'
```

Oggetti dell'ambiente (host-defined)

Ogni **ambiente di esecuzione** (host) di JavaScript può fornire ulteriori nomi predefiniti nello scope globale, con funzioni adatte allo specifico ambiente.

Per esempio, quando JavaScript è eseguito all'interno di una pagina web, sono definite le variabili globali **document** (la pagina web), **window** (la tab del browser), e metodi di utilità per accedere alle risorse di rete.

Alcuni esempi di host:

- Browser web
 - Document, window, location, ...
<https://developer.mozilla.org/en-US/docs/Web/API>
- AppScript (il linguaggio delle app Google)
 - CalendarApp, DriveApp, GmailApp, ...
<https://developers.google.com/apps-script>
- Qt (toolkit applicazioni per desktop)
- Android (quello dei telefonini)
 - Oggetti Java “iniettati” dall'app
- Node.js (esecuzione su server, REPL)
 - console, process, ...
<https://nodejs.org/api/globals.html>

Modulo **fs** (File System)

Contiene funzioni per

- leggere e scrivere contenuti di un file
- leggere i contenuti di una directory
- creare, cancellare, troncare, rinominare, copiare file e directory
- ottenere e impostare *metadati* su file e directory

Funzioni di utilizzo più comune:

- `dati=readFileSync(path)`
- `writeFileSync(path, dati)`

Dove il path è il 'nome' del file, di solito come stringa, mentre i dati possono essere di vario tipo (spesso: stringhe, Buffer o Typed array)

Modulo **fs** (File System)

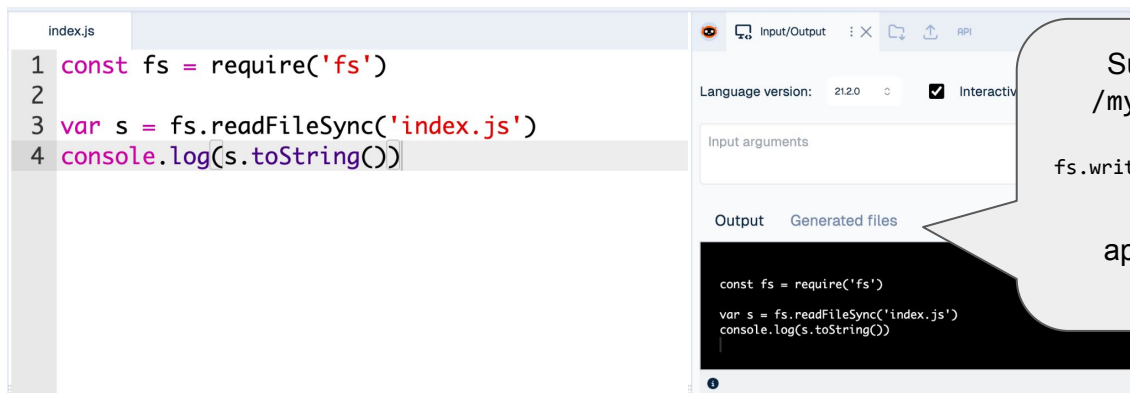
Contiene funzioni per

- leggere e scrivere contenuti di un file
- leggere i contenuti di una directory
- creare, cancellare, troncare, rinominare, copiare file e directory
- ottenere e impostare *metadati* su file e directory

Funzioni di utilizzo più comune:

- `dati=readFileSync(path)`
- `writeFileSync(path, dati)`

Dove il path è il 'nome' del file, di solito come stringa, mentre i dati possono essere di vario tipo (spesso: stringhe, Buffer o Typed array)



The screenshot shows the JDoodle interface. On the left, a code editor displays the following JavaScript code in a file named 'index.js':

```
1 const fs = require('fs')
2
3 var s = fs.readFileSync('index.js')
4 console.log(s.toString())
```

On the right, the 'Output' panel shows the same code being executed, resulting in the same text being printed to the console. The 'Generated files' panel is also visible but empty.

Su JDoodle, i file scritti in /myfiles, ad esempio con:

```
fs.writeFileSync("/myfiles/test.txt",  
"Contenuto file")
```

appaiono poi nel pannello
Generated files

Modulo **fs** (approccio asincrono con callback)

Le funzioni `readFileSync()` e `writeFileSync()` mettono l'intero contenuto del file in una sola stringa: non adatto per file molto grandi!

Altre funzioni consentono di leggere (o scrivere) file molto grandi, *processando* il contenuto in blocchi più piccoli (chunk).

Man mano che la lettura/scrittura procede, una funzione passata come argomento viene invocata ripetutamente, passando un chunk dopo l'altro.

Analogia con `.forEach()` e generatori...

Asincrono: i callback vengono chiamati **nel futuro e fuori ordine**
(man mano che arrivano i dati)

```
const fs=require('fs')

var s=fs.readFileSync('dante.txt')
console.log(1, s.toString().substr(0,100))

fs.readFile('/usr/local/bin/node',
  (err,data) => console.log(2, err, data)
)

fs.readFile('dante.txt','utf-8',
  (err,data) => console.log(3, err, data.substr(0,100))
)
```

1 INFERNO

Inferno • Canto I

Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscu
Hint: hit control+c anytime to enter REPL.

3 null INFERNO

Inferno • Canto I

Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscu
2 null <Buffer 7f 45 4c 46 02 ... 48914830 more bytes>

Modulo `os` (Operating System)

Contiene funzioni per ottenere informazioni sulla macchina, sul sistema operativo, e sull'utente che sta eseguendo il nostro programma.

In particolare, le funzioni `os.hostname()` e `os.networkInterfaces()` forniscono informazioni sugli *indirizzi di rete* che posso usare per accedere alla macchina su cui gira il programma

`os.userInfo()` vi dice “chi siete voi” (per il sistema operativo).

```
➤ os.platform()
'linux'
➤ os.arch()
'x64'
➤ os.totalmem()
27335303168
➤ os.cpuinfo()
[
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2299,
    times: {
      user: 19728040,
      nice: 1270910,
      sys: 12175820,
      idle: 19332080,
      irq: 0
    }
  },
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2299,
    times: {
      user: 19958690,
      nice: 1234020,
      sys: 12073080,
      idle: 19191540,
      irq: 0
    }
  },
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2299,
    times: {
      user: 19451660,
      nice: 1076020,
      sys: 12216260,
      idle: 19658270,
      irq: 0
    }
  },
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2299,
    times: {
      user: 19442060,
      nice: 1051460,
      sys: 12172500,
      idle: 19548570,
      irq: 0
    }
  }
]
➤
```

Modulo process

Fornisce funzioni per vari scopi:

- Ottenere informazioni sul *processo* che sta eseguendo il vostro programma
 - es.: con che *argomenti da riga di comando* è stato lanciato
 - es.: quali sono i valori delle *variabili di ambiente* del sistema operativo
- Manipolare gli altri processi in esecuzione sulla macchina
- Registrare funzioni da eseguire in momenti specifici
 - es.: subito prima di terminare

```
const process=require('process')
console.log(process.env)
```



```
{
  "NODE_VERSION": "12.21.0",
  "HOSTNAME": "7a22f6922a3e",
  "YARN_VERSION": "1.22.5",
  "HOME": "/home/runner",
  "PATH": "/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
  "NODE_PATH": "/usr/local/lib/node_modules:/home/runner/node_modules",
  "PWD": "/home/runner",
  "REPL_PUBKEYS": '{"croasis-ci-2021-03-15":"7YlpcYh82cR9NSTStSYtR5jDL4onNzCGJGq6b+9CuZiI=", "primary-2021-02-08":"tGsJlu/BjWvTgvMaX7acuUb7A0IdXORiuk7yO83RFE="}',
  "REPL_IT_DB_URL": 'https://kv.repl.it.com/v0/eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJleHAiOjEzMTYyYmTKzMtKsImldhCi6MTYxNjEONzcxcOSwianXNZjoiY29ubWVuIiwiaWF0IjE1ZWZGFOWWJhc2VfaWQioiIzZDkzMjUxYS0zMdG4LTQ3MzAtODIzOC03YmNmOXRWRjY2ZmYTYYtfQ.mFq-cWO9MtGXkj8K4Cj-TGz6wa5lYzs_gdMtOXRLYqDYOHmK2RPBEBLeCuxd_FLeHuV8MviUlFkUvrOpVVKdy3g',
  "REPL_ID": "3d93251a-3088-4730-8238-7bcb9dcccffa6",
  "REPL_LANGUAGE": "nodejs",
  "REPL_SLUG": "Lez-36-Scrapboard",
  "REPL_OWNER": "VINCEZOGERVASI",
  "REPL_IMAGE": "gcr.io/marine-cycle-160323/nodejs:106552f39be5dd263f3ef62c4059203cff712fcb",
  "TERM": "xterm-256color"
}
```


Moduli `url` e `http` (accesso al web)

I moduli `url` e `http` forniscono, rispettivamente, funzioni per fare il parsing di **URL** (conversioni URL-stringa e simili) e per utilizzare il **protocollo di rete HTTP**.

In particolare, con HTTP potete implementare facilmente un vostro server web:

- Può restituire i contenuti di un **file**
- Ma anche **calcolare** una risposta e restituirla

Il funzionamento del modulo `http` è troppo complesso per spiegarlo in maniera compiuta, ma possiamo vedere direttamente un esempio:

```
const http=require('http')

var server = http.createServer(function (req, res) {
  console.log(req)
  res.writeHead(200, {'Content-Type': 'text/plain; charset=utf-8'})
  if (req.url.match(/dante/i)) {
    res.end(fs.readFileSync('dante.txt','utf-8'))
  } else {
    res.end("Circolare, non c'è niente da vedere!","utf-8")
  }
})

server.listen(1337);
```

Moduli `url` e `http` (accesso al web)

Accedere al web per ottenere i contenuti corrispondenti a una URL è ancora più facile.

In questo esempio, useremo l'approccio **asincrono** e registreremo due **eventi** corrispondenti alla ricezione di un chunk di dati e alla fine della trasmissione.

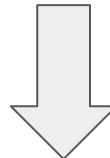
Abbiamo tempo solo per un esempio... ma l'approccio ad **eventi** è tipico di come si gestiscono in maniera complessa gli eventi asincroni:

- Operazioni su disco
- Comunicazioni di rete
- Input dell'utente (mouse, tastiera, ...)

```
const http=require('http')

let req=http.get("http://www.di.unipi.it", 'utf-8', res=>{
  var pag=''
  res.on('data', chunk=>{pag+=chunk})
  res.on('end', ()=>{console.log(pag)})
})
req.end()
```

Questo modo di intercettare eventi (metodi `on(evento, callback)`) è detto **modello a stream**



```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
<p>The document has moved <a href="https://www.di.unipi.it/">here</a>.</p>
<hr>
<address>Apache Server at www.di.unipi.it Port 80</address>
</body></html>
```

Q & A